VSCode + Copilot + Claude + Figma 协作开发指南

> 版本: v1.0
>
> 日期: 2026-04-10
>
> 适用项目: PropOS Phase 1
>
> 适用角色: 全栈开发者 / 前端开发者

目录

一、四大工具定位与协作全景

- 1.1 工具角色定义
- 1.2 协作全景图
- 1.3 核心原则

二、环境配置与工具安装

- 2.1 VSCode 扩展清单
- 2.2 VSCode 配置
- 2.3 Figma 配置
- 2.4 Copilot 模型选择

三、核心工作流：从设计到代码

- 3.1 工作流总览
- 3.2 Phase A：骨架生成（Day 0~2）
- 3.3 Phase B：Figma 设计（Day 3~10）
  - 设计师需要从开发者获得的输入
  - 设计师在 Figma 中的工作
- 3.4 Phase C：视觉接入（Day 11~14）

四、Figma → 代码：三种桥接方法

- 方法 1：Design Token 表驱动（推荐 ⭐⭐⭐）
- 方法 2：Figma Dev Mode + CSS 复制（推荐 ⭐⭐）
- 方法 3：截图 + 文字描述（适用于新组件 ⭐）
- 三种方法对比

五、Copilot + Claude 编码实战

- 5.1 三种对话模式选择
- 5.2 上下文管理要点
- 5.3 常用提示词模板
  - 从 Figma 生成新页面

对齐已有页面到 Figma 设计
批量应用 Design Token
5.4 Copilot 定制化体系（已内置）
六、全栈开发日常节奏
6.1 典型一天
6.2 模块开发全流程示例（以 M2 合同管理为例）
6.3 各模块推荐工作流变体
七、技巧与避坑指南
7.1 Figma 侧技巧
7.2 Copilot 侧技巧
7.3 常见问题与解决
7.4 绝对禁止事项
附录：快速参考卡片
A. Figma → Code 速查
B. Copilot 模式速查
C. 提示词速查
D. 关键文档引用速查

一、四大工具定位与协作全景

1.1 工具角色定义

工具	角色	擅长	不擅长
VSCode	主战场	代码编辑、终端、调试、Git、扩展生态	视觉设计
Copilot (Claude)	AI 编程搭档	代码生成、重构、审查、文档、多文件编排	像素级 UI 还原、直接读 Figma
Claude (Copilot 内置)	推理引擎	复杂业务逻辑、架构设计、长上下文理解	实时访问外部服务
Figma	视觉设计工具	UI/UX 设计、Design Token 管理、交互原型	生成可用的生产代码

1.2 协作全景图

```
┌──────────────────────────────────────────────────────────────────┐
│                          开发者工作台                            │
│                                                                  │
│  ┌──────────┐    Design Token / 截图 / CSS    ┌──────────┐       │
│  │  Figma   │ ──────────────────────────────→ │  VSCode  │       │
│  │ (设计稿) │                                 │ (代码编辑) │       │
```

```
│   └──────┘                                  └──────┘            │
│        │                                          │             │
│        │  Dev Mode                          Copilot Chat / Agent  │
│        │  查看 CSS 属性          代码生成 + 审查     │
│        │  导出切图                 │                │
│        │                                          ▼             │
│        │                                   ┌──────────────┐     │
│        └──────────── 视觉验收 ──────────→  │ Claude (AI)  │     │
│                                            │ 理解设计意图 │     │
│                                            │ 生成组件代码 │     │
│                                            └──────────────┘     │
└──────────────────────────────────────────────────────────────────┘
```

1.3 核心原则

> **Figma 负责「设计」，Copilot (Claude) 负责「代码」，VSCode 负责「执行」，三者通过 Design Token 和截图作为桥梁。**

二、环境配置与工具安装

2.1 VSCode 扩展清单

扩展	用途	必装
GitHub Copilot	AI 代码补全	✅
GitHub Copilot Chat	AI 对话（Agent/Plan/Ask）	✅
Dart	后端语言支持	✅
Flutter	Flutter SDK 集成、热重载、Widget 检查	✅
Vue - Official (Volar)	Vue 3 + TypeScript 支持	✅
ESLint	Admin 前端代码规范	✅
Figma for VS Code	在 VSCode 侧栏预览 Figma 设计稿	推荐

2.2 VSCode 配置

在 `.vscode/settings.json` 中确保以下配置：

```
{
  "github.copilot.chat.codeGeneration.useInstructionFiles": true,
  "github.copilot.nextEditSuggestions.enabled": true,
  "chat.agent.enabled": true,
  "github.copilot.enable": {
    "*": true,
```

```
    "dart": true
  }
}
```

2.3 Figma 配置

配置项	说明
Figma 账号	免费版即可（Dev Mode 需付费，但可用 14 天试用）
项目文件结构	按模块建页面：`M1-Assets` / `M2-Contracts` / `M3-Finance` / `M4-Workorders` / `M5-Sublease`
组件库	使用 Element Plus Figma Kit（PC 端）+ Material 3 Design Kit（移动端）
插件推荐	`Tokens Studio`（Design Token 管理）、`Iconify`（图标）、`Content Reel`（填充数据）

2.4 Copilot 模型选择

在 Copilot Chat 中确认使用 **Claude** 模型（下拉选择）。Claude 在以下场景优于其他模型：

- 长上下文理解（PropOS 的 PRD 约 8000 字，Claude 可完整理解）
- 复杂业务逻辑推理（WALE 公式、递增规则引擎）
- 中文场景（PropOS 全中文 UI + 文档）

三、核心工作流：从设计到代码

3.1 工作流总览

PropOS 采用「骨架先行，设计并行」策略（详见 `DEV_UI_SYNC_GUIDE.md` ），Figma 与代码的协作流程如下：

```
Day 0~2   骨架生成    | VSCode + Copilot 生成页面骨架 + MockData
Day 3     对齐日      | 截图/录屏 → 设计师在 Figma 中理解数据密度
Day 4~10  双线并行    | 开发完善逻辑 ↔ 设计师在 Figma 出高保真稿
Day 11~14 视觉接入    | Figma Design Token → Material 3 ThemeData / Element Plus 主题 → 视觉对齐
```

3.2 Phase A：骨架生成（Day 0~2）

工具组合：VSCode + Copilot (Agent 模式)

1. 在 Copilot Chat 切换到 **Agent** 模式
2. 引用项目文档作为上下文
3. 使用内置提示词命令生成页面骨架

```
# 示例：生成资产总览页面骨架
@PAGE_WIREFRAMES_v1.8.md @PAGE_SPEC_v1.8.md

根据线框文档中的"资产总览"页面描述，生成 admin/src/views/assets/AssetOverview.vue 页面骨架，
```

```
使用 Element Plus 组件库，包含 MockData 展示三业态概览数据。
```

3.3 Phase B：Figma 设计（Day 3~10）

工具组合：Figma（设计师主导）+ VSCode 截图输入

设计师需要从开发者获得的输入

输入	格式	来源
页面骨架截图	PNG（iPhone 14 Pro + 1920×1080）	开发者真机 + 浏览器截图
操作流程录屏	MP4	开发者录制核心流程
状态色彩语义	表格	`DEV_UI_SYNC_GUIDE.md` §5.2
数据边界速查	表格	`DEV_UI_SYNC_GUIDE.md` §5.3
组件 Props 签名	Dart 接口 / TypeScript 接口	开发者在骨架阶段确定

设计师在 *Figma* 中的工作

1. 参照骨架截图确认数据密度和布局合理性
2. 使用 **Tokens Studio** 插件定义 Design Token（颜色/字体/间距）
3. 输出高保真稿，标注交互细节

3.4 Phase C：视觉接入（Day 11~14）

工具组合：Figma (Dev Mode) → VSCode + Copilot

这是四大工具协作最密集的阶段，详见下一节。

四、Figma → 代码：三种桥接方法

由于 Copilot 无法直接读取 Figma 文件，需要开发者手动桥接。以下三种方法按推荐度排序：

方法 1：Design Token 表驱动（推荐 ⭐⭐⭐）

> 最可靠、最可维护的方法。设计变更只改 Token 值，不改组件代码。

步骤：

1. 设计师在 Figma 使用 `Tokens Studio` 插件管理 Design Token
2. 导出 Token 为 JSON 或 Markdown 表格
3. 在 Copilot Chat 中提交 Token 表，让 AI 生成 CSS 变量

```
将以下 Design Token 写入 admin/src/styles/variables.scss 和 flutter_app/lib/core/theme/
app_theme.dart：

颜色 Token：
```

```
| --color-primary        | #1976D2 | 主品牌色 |
| --color-success        | #388E3C | 已租/已核销 |
| --color-danger         | #D32F2F | 空置/逾期 |
| --color-warning        | #F57C00 | 即将到期/预警 |

字体 Token:
| --font-display-large   | 32px Bold      | NOI 大数字 |
| --font-body-medium     | 14px Regular   | 列表行内容 |

间距 Token:
| --spacing-page         | 24px | 页面内边距 |
| --spacing-card         | 16px | 卡片内边距 |
```

Copilot 会生成:

- `admin/src/styles/variables.scss` — Element Plus 主题覆盖
- `flutter_app/lib/core/theme/app_theme.dart` — Flutter Material 3 ThemeData + ThemeExtension

方法 2：Figma Dev Mode + CSS 复制（推荐 ⭐⭐）

> 适合精调单个组件的间距、圆角、阴影等属性。

步骤:

1. 在 Figma 中切换到 **Dev Mode**（Inspect 面板）
2. 选中目标元素，复制右侧面板中的 CSS 属性
3. 粘贴到 Copilot Chat，让 AI 应用到对应组件

```
将以下 Figma 导出的 CSS 属性应用到 admin/src/views/dashboard/DashboardView.vue 中的 NOI 卡片组件:

/* Figma Dev Mode 导出 */
border-radius: 12px;
box-shadow: 0px 2px 8px rgba(0, 0, 0, 0.08);
padding: 24px;
background: #FFFFFF;
gap: 16px;

保持 Element Plus 的 el-card 组件结构不变，通过 scoped CSS 覆盖样式。
```

方法 3：截图 + 文字描述（适用于新组件 ⭐）

> 适合从零创建一个 Figma 稿中的新组件，精度有限但速度快。

步骤:

1. 将 Figma 设计稿的目标区域导出为 PNG
2. 在 Copilot Chat 中贴入截图 + 文字补充

```
参照附图中的合同详情页设计，生成 admin/src/views/contracts/ContractDetail.vue。

设计要点:
```

```
- 顶部：合同编号 + 状态 Tag（使用 el-tag type 对应状态色）
- 左侧 60%：合同基本信息（el-descriptions 组件，2 列布局）
- 右侧 40%：租金递增时间线（el-timeline 组件）
- 底部 Tab 页：关联单元 / 账单列表 / 子租赁（使用 el-tabs）
- 卡片圆角 12px，间距参照 Design Token
```

> 注意：截图方法的精度依赖文字描述的详细程度。色值、间距等精确参数务必用文字补充，不要完全依赖 AI 从截图中推断。

三种方法对比

方法	精度	速度	适用阶段	可维护性
Design Token 表驱动	⭐⭐⭐	⭐⭐	全局主题/全量页面	⭐⭐⭐
Dev Mode CSS 复制	⭐⭐⭐	⭐⭐⭐	单组件精调	⭐⭐
截图 + 描述	⭐	⭐⭐⭐	从零创建新组件	⭐

推荐组合：先用方法 1 统一全局 Token，再用方法 2 精调关键组件，最后用方法 3 快速出新组件原型。

五、Copilot + Claude 编码实战

5.1 三种对话模式选择

模式	触发方式	最佳场景	Figma 相关用法
Agent	Copilot Chat 顶部切换	多文件生成/修改、全栈模块实现	根据 Design Token 批量更新所有页面样式
Plan	Copilot Chat 顶部切换	定向修改、确认后执行	单个组件的 Figma CSS 对齐
Ask	Copilot Chat 顶部切换	只读问答、设计方案讨论	"这个 Figma 布局用什么 Element Plus 组件实现？"

5.2 上下文管理要点

规则	说明
每次对话引用关键文件	`@PRD.md` `@ARCH.md` `@PAGE_WIREFRAMES_v1.8.md`
一次对话聚焦一个模块	不在同一对话中混合 M1 和 M3
先确认数据结构再写代码	让 AI 先输出类型定义，确认后再生成实现
复杂逻辑分步生成	递增引擎、KPI 打分等拆成多步，逐步验证

5.3 常用提示词模板

从 *Figma* 生成新页面

```
@PAGE_SPEC_v1.8.md

根据 PAGE_SPEC 中"[页面名称]"的描述，生成 [admin/flutter_app] 端页面。

Figma 设计参数：
- 布局：[描述 Figma 中的布局结构]
- 状态色：Flutter 使用 Theme.of(context).colorScheme.* / Admin 使用 CSS 变量 --color-success / --color-warning / --color-danger
- 间距：页面 24px，卡片 16px，列表项 12px
- 字体：标题 --font-headline-large，正文 --font-body-medium

要求遵循 api → store → page 单向数据流。
```

对齐已有页面到 *Figma* 设计

```
调整 [文件路径] 的样式，使其对齐以下 Figma 设计规范：

[粘贴 Figma Dev Mode 的 CSS 属性]

保持业务逻辑不变，只修改样式部分（Flutter：Widget 属性；Admin：`<style scoped>`）。
不要更改组件结构和 Props。
```

批量应用 *Design Token*

```
将以下 Design Token 更新同步到项目中：

[粘贴 Token 表格]

需要修改的文件：
1. flutter_app/lib/core/theme/app_theme.dart — Material 3 ThemeData + ThemeExtension
2. admin/src/styles/variables.scss — Element Plus 主题变量
3. 所有使用硬编码色值的组件（Flutter：搜索 `Colors.` / `Color(0xFF`；Admin：搜索 `style="color:"` 替换为 CSS 变量）
```

5.4 Copilot 定制化体系（已内置）

PropOS 项目已配置完整的四层 Copilot 定制化体系，**自动生效无需额外操作**：

层级	文件	作用
Layer 1	`copilot-instructions.md`	全局规则，所有对话自动加载
Layer 2	`instructions/*.instructions.md`	编辑匹配路径时自动注入（如 Repository 层 SQL 规范）
Layer 3	`prompts/*.prompt.md`	`/` 命令手动调用（如 `/backend-module`）

Layer 4	`agents/*.agent.md`	Agent 选择器调用（如 PropOS Feature Builder）

> 关键意义：当你让 Copilot 生成代码时，它已自动知晓 PropOS 的三业态模型、RBAC 规则、API 信封格式、常量管理策略等约束，无需每次重复说明。

六、全栈开发日常节奏

6.1 典型一天

```
09:00  ┌── 查看 Figma 更新通知，确认设计师昨天是否有新稿产出
       │
09:15  ├── VSCode Agent 模式：按计划实现后端模块
       │    （Model → Repository → Service → Controller）
       │
11:00  ├── Figma Dev Mode：复制新组件的 CSS 参数
       │   → Copilot Plan 模式：对齐前端组件样式
       │
12:00  ├── 午休
       │
13:00  ├── VSCode Agent 模式：实现前端 Store + 组件
       │   引用 @PAGE_WIREFRAMES 确保对齐线框
       │
15:00  ├── 真机/浏览器测试，截图发设计师确认
       │   → 若有偏差，Copilot Plan 模式微调
       │
16:00  ├── Copilot Ask 模式：规划明天的任务
       │   评审今天写的代码
       │
17:00  └── 更新文档 → bash scripts/md_to_pdf.sh
```

6.2 模块开发全流程示例（以 M2 合同管理为例）

```
Step 1  Copilot Ask：讨论递增规则的数据结构设计
                ↓
Step 2  Copilot Agent：生成 Dart sealed class + 计算引擎 + 单元测试
                ↓
Step 3  Copilot Agent：生成后端四层（Repository → Service → Controller）
                ↓
Step 4  Copilot Agent：使用 /flutter-page 生成移动端页面骨架
                ↓
Step 5  Copilot Agent：使用 /admin-view 生成 PC 端页面骨架
                ↓
Step 6  截图/录屏 → 发给设计师 → 设计师在 Figma 出稿
                ↓
Step 7  Figma Dev Mode → 复制 CSS → Copilot Plan 对齐样式
```

```
                    ↓
Step 8   设计师提交 Design Token → Copilot Agent 批量更新 ThemeData / CSS 变量
                    ↓
Step 9   真机验收 → 交付
```

6.3 各模块推荐工作流变体

模块	Figma 重度	Copilot 重度	说明
M1 资产可视化	⭐⭐⭐	⭐⭐	楼层热区图视觉要求高，需 Figma 精调
M2 租务合同	⭐⭐	⭐⭐⭐	递增引擎等计算逻辑复杂，Copilot 主导
M3 财务 NOI	⭐⭐⭐	⭐⭐⭐	看板图表视觉要求高 + 计算逻辑复杂
M4 工单系统	⭐⭐	⭐⭐	移动端标准 CRUD，两端工作量均衡
M5 二房东穿透	⭐	⭐⭐⭐	外部门户简洁风格，重点在权限隔离逻辑

七、技巧与避坑指南

7.1 Figma 侧技巧

技巧	说明
组件命名与代码对齐	Figma 中组件命名用 PascalCase（如 `ContractCard`），与 Vue 组件名一致，方便沟通
Auto Layout 对应 Flex	Figma 的 Auto Layout 直接对应 CSS Flexbox，属性名几乎一一映射
使用 Variants 标注状态	用 Figma Variants 表达组件不同状态（如 `state=leased/vacant`），对应 Flutter Widget props 或 Vue props
导出切图用 **2x**	图标和装饰图导出 SVG 或 2x PNG，在 VSCode 中直接使用
Tokens Studio 导出 JSON	可直接导出为 JSON 格式，Copilot 可解析并生成对应 CSS 变量文件

7.2 Copilot 侧技巧

技巧	说明
截图贴入对话	直接在 Copilot Chat 中粘贴 Figma 截图，AI 能识别布局结构
引用线框文档	`@PAGE_WIREFRAMES_v1.8.md` 比截图更精确，优先用文字描述
先 **Token** 后组件	先让 AI 理解 Design Token 体系，再生成具体组件，避免硬编码色值
分层提示	"只修改样式不改逻辑" 或 "只改 Store 不改组件"，避免 AI 过度修改
使用内置 **Agent**	PropOS Feature Builder 自动按正确顺序生成全栈代码

7.3 常见问题与解决

问题	原因	解决方案
Copilot 生成的样式与 Figma 不一致	AI 无法精确读图	不依赖截图推断数值，用 Figma Dev Mode 复制精确 CSS
组件结构与设计稿有出入	信息不对称	先确认组件 Props 接口，再出 Figma 稿
Design Token 更新后部分页面未生效	仍有硬编码色值	Flutter: 检查是否有 `Colors.xxx` / `Color(0xFF...)`；Admin: `grep -r "color:" --include="*.vue"` 排查并替换
Figma 中文字体与实际渲染不同	系统字体差异	Figma 和代码统一使用 `system-ui, -apple-system` 字体栈
Element Plus 组件样式覆盖失败	CSS 优先级不够	使用 `:deep()` 穿透 + CSS 变量覆盖，避免 `!important`

7.4 绝对禁止事项

禁止	原因
❤ 在组件中硬编码色值	Flutter 禁止 `Colors.green` / `Color(0xFF...)`，必须用 `Theme.of(context).colorScheme.*`；Admin 禁止 `style="color: #388E3C"`，必须用 CSS 变量
❤ 设计师直接修改源代码	破坏代码分层约定
❌ 开发者绕过 Figma 自行"美化"	产生设计漂移，难以维护
❌ 用 Figma 预览验收替代真机测试	Figma 不模拟真实屏幕密度和字体渲染
❌ 更改状态色彩语义（如把空置改成蓝色）	🟢已租/🟡即将到期/🔴空置/⚪非可租是业务约束
❌ 让 Copilot 一次生成整个模块（5+ 文件）	分步生成，逐步验证质量更可控

附录：快速参考卡片

A. Figma → Code 速查

```
需要全局主题 → Design Token JSON/表格 → Copilot 生成 ThemeData / CSS 变量 [方法1]
需要精调组件 → Figma Dev Mode 复制 CSS → Copilot Plan 模式应用 [方法2]
需要新建组件 → Figma 截图 + 文字描述 → Copilot Agent 模式生成 [方法3]
```

B. Copilot 模式速查

```
多文件生成/骨架搭建   → Agent 模式
定向修改/样式对齐     → Plan 模式
方案讨论/代码评审     → Ask 模式
全栈模块按序实现      → Agent 选择器 → PropOS Feature Builder
```

C. 提示词速查

```
/backend-module     → 后端完整模块（4 层）
/flutter-page       → Flutter 页面（实体 + API + BLoC/Cubit + 页面）
/admin-view         → admin 页面（类型 + API + Store + 视图）
/security-and-test  → 安全审查 + 测试计划
```

D. 关键文档引用速查

```
@PRD.md                      → 需求定义（所有业务规则的权威来源）
@ARCH.md                     → 技术架构
@PAGE_WIREFRAMES_v1.8.md     → 页面线框（文字版 Figma）
@PAGE_SPEC_v1.8.md           → 页面功能规格
@DEV_UI_SYNC_GUIDE.md        → 开发与设计同步流程
@COPILOT_GUIDE.md            → Copilot 深度使用指南
```

> **延伸阅读**
>
> - Copilot + Claude 深度使用指南 → `COPILOT_GUIDE.md`
> - 开发与 UI 设计同步指南 → `DEV_UI_SYNC_GUIDE.md`
> - 开发环境搭建 → `DEV_ENV_SETUP.md`

本文档随工具版本和项目实践持续更新。